

Федеральное государственное автономное  
образовательное учреждение  
высшего образования  
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»  
Институт космических и информационных технологий  
Кафедра «Вычислительная техника»

**Основы проектирования программного обеспечения**

Инструментальные средства анализа кода программ

Преподаватель

Студент КИ24-06Б, 032429826  
номер группы, зачетной книжкой

\_\_\_\_\_  
подпись, дата

\_\_\_\_\_  
подпись, дата

А. П. Яблонский

инициалы, фамилия

Р. В. Васильев

инициалы, фамилия

Красноярск 2025

## **Введение**

В качестве объекта исследования используется программа из практической работы №3 для обработки двумерных точек (координаты, цвет). В работе №3 программа была рефакторингована и покрыта модульными тестами Pytest. Цель текущей работы - оценить качество решения с применением внешних инструментов анализа и устранить выявленные недостатки.

Работа заключается в:

1. исследовании программы с использованием инструментов статического и динамического анализа кода, не встроенных в IDE по умолчанию;
2. оценке качества решения по критериям качества ПО;
3. рефакторинге и устранении выявленных дефектов;
4. оформлении UML-диаграммы зависимостей.

## **Исходный код программы (работа 3)**

Исходный код работы №3 включает следующие основные файлы проекта:

1. `Points_Table_VasilevR2.py` – реализация функций и консольного интерфейса;
2. `Test_points_pytest.py` – модульные тесты Pytest для нетривиальной логики.

Полная версия проекта и история изменений размещаются в репозитории

<https://github.com/honexxx/OPPO>

## **Статический анализ кода**

Для статического анализа был установлен комплексный статический анализатор Python кода Prospector, который содержит в себе несколько инструментов анализа (`dodgy`, `mccabe`, `profile-validator`, `pycodestyle`, `pyflakes`, `pylint`). Они предназначены для проверки стиля, анализа логических ошибок, проверки соответствия PEP 8Б, измерения цикломатической сложности, поиска потенциальных угроз безопасности и проверки совместимости с профилями. Пример команды: `prospector Points_Table_Vasilev_R2.py --strictness veryhigh > prospector_report.txt`

Результат анализа:

```
1 Check Information
2 =====
3     Started: 2026-01-13 00:31:05.491220
4     Finished: 2026-01-13 00:31:06.131469
5     Time Taken: 0.64 seconds
6     Formatter: grouped
7     Profiles: .prospector.yaml, default, strictness_medium, strictness_high, strictness_veryhigh, no_doc_warnings, no_test_warnings, no_member_warnings
8     Strictness: veryhigh
9     Libraries Used: django
10    Tools Run: dodgy, mccabe, profile-validator, pycodestyle, pyflakes, pylint
11    Messages Found: 0
```

Рисунок 1 – Тест Prospector

По результатам статического тестирования можно сделать вывод, что критических проблем не обнаружено.

## Динамический анализ кода

Динамический анализ выполнен с помощью модульных тестов Pytest.

```
pytest test_points_pytest.py -v
===== test session starts =====
platform win32 -- Python 3.13.9, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\Роман\PycharmProjects\0ppo1\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Роман\PycharmProjects\0ppo1
collected 19 items

test_points_pytest.py::test_valid_input_numbers_first PASSED [ 5%]
test_points_pytest.py::test_valid_input_color_first PASSED [ 10%]
test_points_pytest.py::test_invalid_color PASSED [ 15%]
test_points_pytest.py::test_missing_color PASSED [ 21%]
test_points_pytest.py::test_extra_numbers PASSED [ 26%]
test_points_pytest.py::test_negative_numbers PASSED [ 31%]
test_points_pytest.py::test_missing_numbers PASSED [ 36%]
test_points_pytest.py::test_numbers_with_signs PASSED [ 42%]
test_points_pytest.py::test_distance_from_origin PASSED [ 47%]
test_points_pytest.py::test_distance_to_another_point PASSED [ 52%]
test_points_pytest.py::test_invalid_color_creation PASSED [ 57%]
test_points_pytest.py::test_distance_negative_coords PASSED [ 63%]
test_points_pytest.py::test_sort_by_distance PASSED [ 68%]
test_points_pytest.py::test_filter_by_radius_include_all PASSED [ 73%]
test_points_pytest.py::test_filter_by_radius_include_none PASSED [ 78%]
test_points_pytest.py::test_filter_by_radius_partial PASSED [ 84%]
test_points_pytest.py::test_sort_same_distance PASSED [ 89%]
test_points_pytest.py::test_valid_colors PASSED [ 94%]
test_points_pytest.py::test_invalid_colors PASSED [100%]

===== 19 passed in 0.03s =====
```

Рисунок 2 - результат выполнения модульных тестов Pytest.

Pytest не показал никаких ошибок, но в ходе динамического анализа было выявлены следующие проблемы: отсутствие обработки исключения `FileNotFoundException` при попытке чтения файла, логическое ограничение - задан цвет центра координаты “red” в классе `Point2D`, при вводе в координату центра любого символа (не цифры), код падал.

## Оценка качества решения

Оценка выполнена по базовым критериям качества ПО.

Таблица 1

| Критерий         | Состояние (до исправлений)                                                                                                                  | Что улучшено                                                                                          |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Надёжность       | Программа аварийно завершалась при отсутствии файла points.txt.<br>Некорректный ввод пользователя (нечисловые данные) приводил к исключению | Добавлена обработка FileNotFoundException.<br>Реализована защита от неверного пользовательского ввода |
| Сопровождаемость | Цвет центра окружности был задан жёстко ("red"), логика расчётов и ограничения модели были неочевидны                                       | Цвет сделан необязательным параметром точки.<br>Логика предметной области стала прозрачнее            |
| Переносимость    | Возможны проблемы с кодировкой на разных системах                                                                                           | Поведение программы зависело от наличия конкретного файла                                             |
| Тестируемость    | Ошибки приводили к аварийному завершению, усложняя тестирование                                                                             | Логика обработки данных изолирована от ввода/вывода                                                   |

**Улучшенный исходный код (после устранения дефектов)**

<https://github.com/honexxx/OPPO>

После анализа кода выполнены следующие точечные улучшения:

1. В функцию main добавлена обработка ситуации, когда входной файл отсутствует.
2. В функцию read\_float добавлена проверка корректности ввода числовых значений.
3. Цвет точки сделан необязательным параметром.
4. В функции \_parse\_line ошибочные строки файла теперь не прерывают обработку остальных данных

Улучшенная версия размещена в репозитории вместе с отчётом.

### UML-диаграмма зависимостей

Рисунок 2 - UML-диаграмма зависимостей компонентов программы.

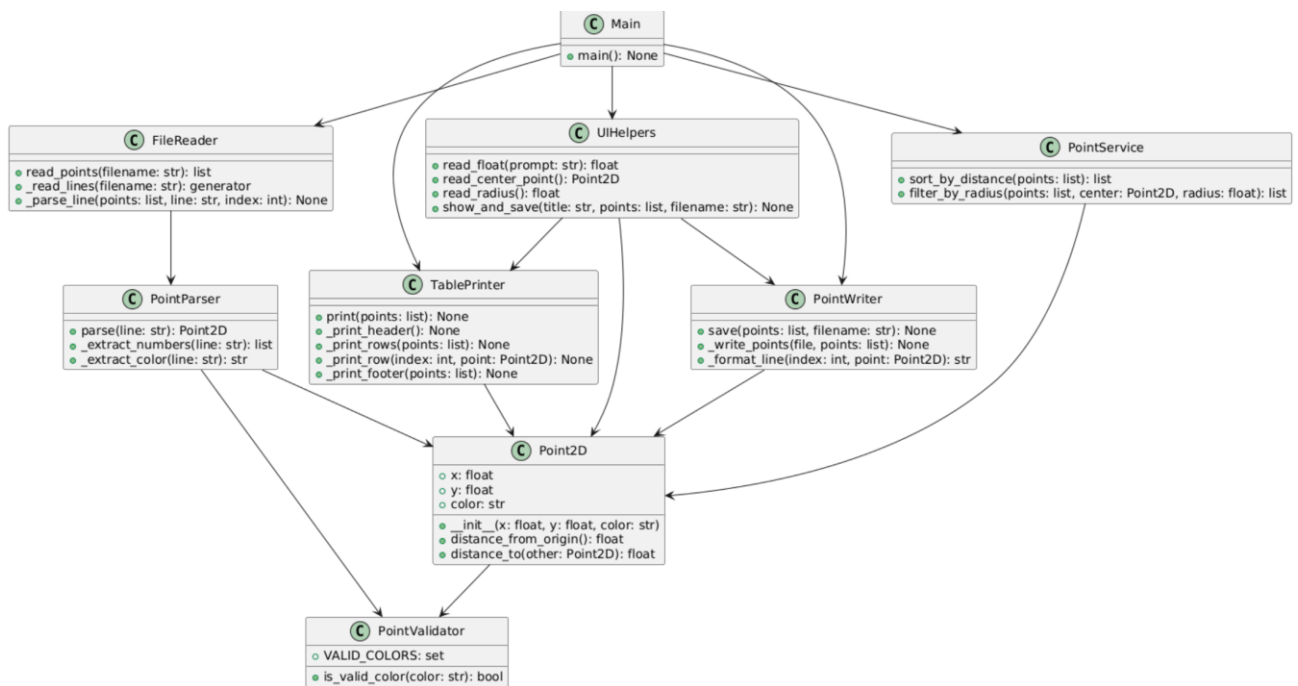


Рисунок 2 – UML-диаграмма

## **Заключение**

В ходе работы выполнен статический и динамический анализ программы из работы №3 с использованием внешних инструментов (Prospector, Pytest).

Выявлены и устранены дефекты, влияющие на переносимость и сопровождаемость. Подготовлена UML-диаграмма зависимостей и оформлен отчёт.